

PARCIAL I CORTE

OSCAR YAIR PARDO PINEDA
JORGE ALFREDO LEAL CRUZ

PRESENTADO A:

ELIECER MONTERO OJEDA

GRUPO: E196

INGENIERIA DE SISTEMAS



INSTITUCION UNIVERSITARIA TECNOLOGICA DE SANTANDER
BUCARAMANGA

2026 – 1

TABLA DE CONTENIDO

INTRODUCCION	3
OBJETIVO GENERAL	4
OBJETIVOS ESPECIFICOS	4
DESCRIPCION DEL PROYECTO	4
DIAGRAMA DE CLASES	5
ARQUITECTURA DEL SISTEMA	6
PATRONES DE DISEÑO DE SOFTWARE.....	12
PATRON SINGLETON	12
IMPLEMENTACION EN EL CODIGO	12
DIAGRAMA UML	14
PRUEBAS	14
PATRÓN FACTORY METHOD.....	15
IMPLEMENTACION EN EL CODIGO	15
DIAGRAMA UML	16
PRUEBAS	17
PATRÓN ABSTRACT FACTORY	19
IMPLEMENTACION EN EL CODIGO	19
DIAGRAMA UML	22
PRUEBAS	23
PATRON BUILDER	26
IMPLEMENTACION EN EL CODIGO	26
DIAGRAMA UML	30
PRUEBAS	31
CONCLUSIONES	33
REFERENCIAS BIBLIOGRAFICAS	35

INTRODUCCION

El desarrollo de software moderno requiere la aplicación de principios de diseño que permitan construir sistemas escalables, mantenibles y desacoplados. En el contexto del sector logístico y de transporte, los sistemas informáticos cumplen un papel fundamental en la gestión eficiente de flotas de vehículos, el monitoreo de operaciones y la optimización de recursos.

El presente proyecto plantea el desarrollo de un Sistema de Gestión de Flotas, orientado a administrar vehículos de transporte, controlar órdenes de operación y facilitar la toma de decisiones dentro de un entorno logístico. El sistema se apoya en una arquitectura hexagonal, la cual permite separar claramente la lógica de negocio de las dependencias tecnológicas externas, favoreciendo la mantenibilidad y evolución del sistema.

Adicionalmente, se implementan diversos patrones de diseño creacionales, tales como Singleton, Factory Method, Abstract Factory y Builder, con el propósito de mejorar la organización del código, reducir el acoplamiento entre componentes y facilitar la creación de objetos complejos dentro del dominio del sistema.

El uso de estos patrones demuestra la aplicación de buenas prácticas de ingeniería de software orientada a objetos, permitiendo construir una solución robusta que simula un entorno real de gestión logística y transporte.

OBJETIVO GENERAL

Desarrollar un Sistema de Gestión de Flotas que permita administrar vehículos y operaciones de transporte mediante el uso de una arquitectura desacoplada y la implementación de patrones de diseño que mejoren la escalabilidad, mantenibilidad y organización del software.

OBJETIVOS ESPECIFICOS

- Diseñar la arquitectura del Sistema de Gestión de Flotas utilizando el modelo de arquitectura hexagonal, con el fin de separar adecuadamente la lógica de negocio, la capa de aplicación y la infraestructura, garantizando un sistema desacoplado y mantenible.
- Implementar los módulos del sistema para la gestión de vehículos, permitiendo realizar operaciones de registro, consulta, actualización y eliminación de información mediante servicios y endpoints que soporten la administración de la flota.
- Evaluar el funcionamiento del sistema mediante pruebas sobre los servicios implementados, verificando la correcta gestión de los vehículos y la interacción entre las diferentes capas de la arquitectura del sistema.

DESCRIPCION DEL PROYECTO

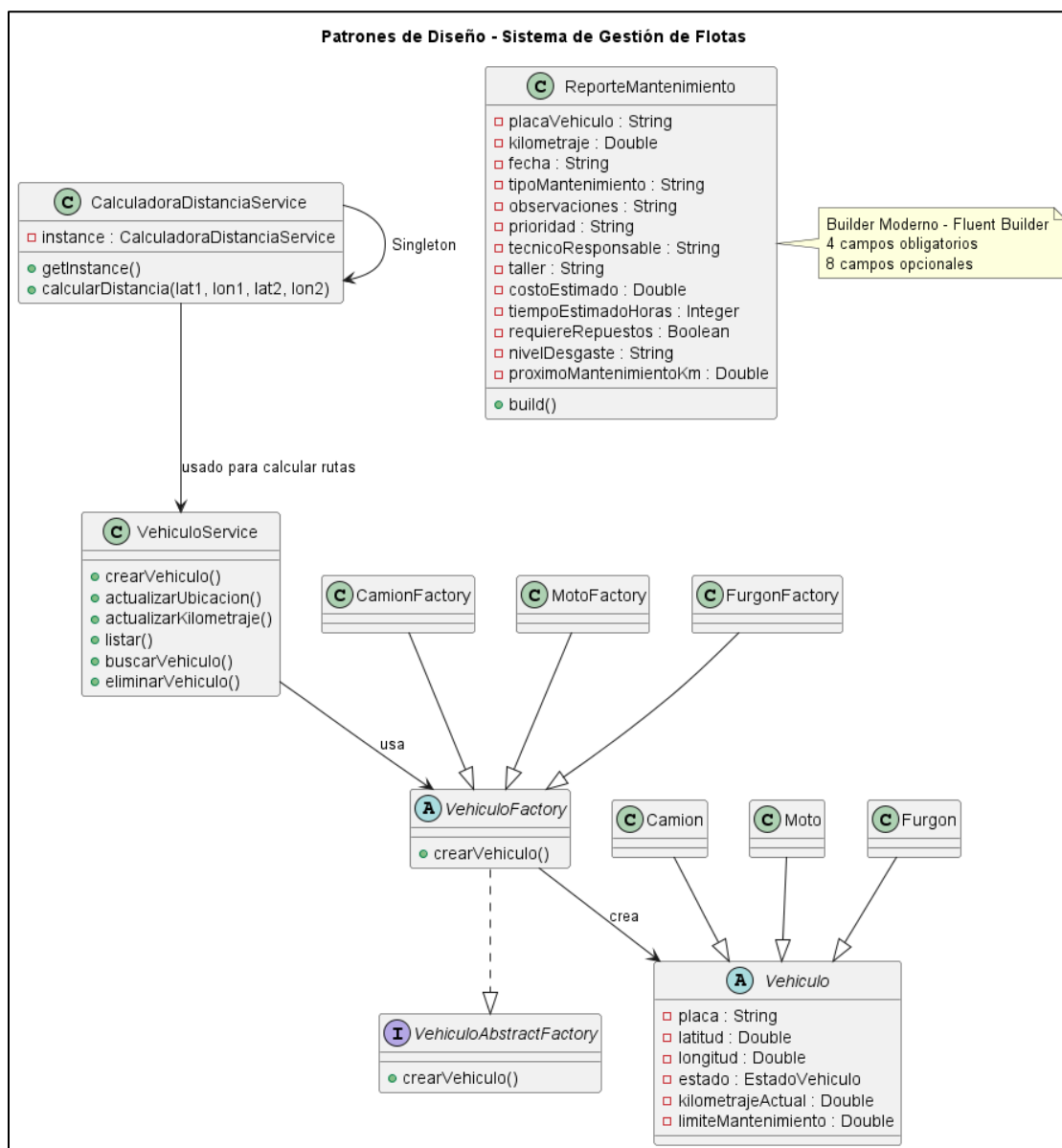


El proyecto consiste en el desarrollo de un Sistema de Gestión de Flotas enfocado en el sector logístico y de transporte, cuyo objetivo es administrar los vehículos que conforman una flota y facilitar la gestión de las operaciones de transporte.

El sistema permite realizar funciones como:

- Monitoreo en tiempo real de los vehículos registrados.
- Optimización de rutas para mejorar la eficiencia del transporte.
- Gestión y asignación de órdenes de transporte.
- Mantenimiento predictivo de los vehículos.
- Integración con sistemas de navegación.

DIAGRAMA DE CLASES



ARQUITECTURA DEL SISTEMA

La Arquitectura Hexagonal, también conocida como Arquitectura de Puertos y Adaptadores, es un modelo arquitectónico propuesto por Alistair Cockburn cuyo objetivo principal es aislar completamente la lógica de negocio de cualquier dependencia tecnológica externa.

Su principio fundamental establece que:

- El núcleo del sistema (dominio) no debe depender de frameworks, bases de datos ni interfaces externas.
- Las dependencias deben apuntar siempre hacia el interior.

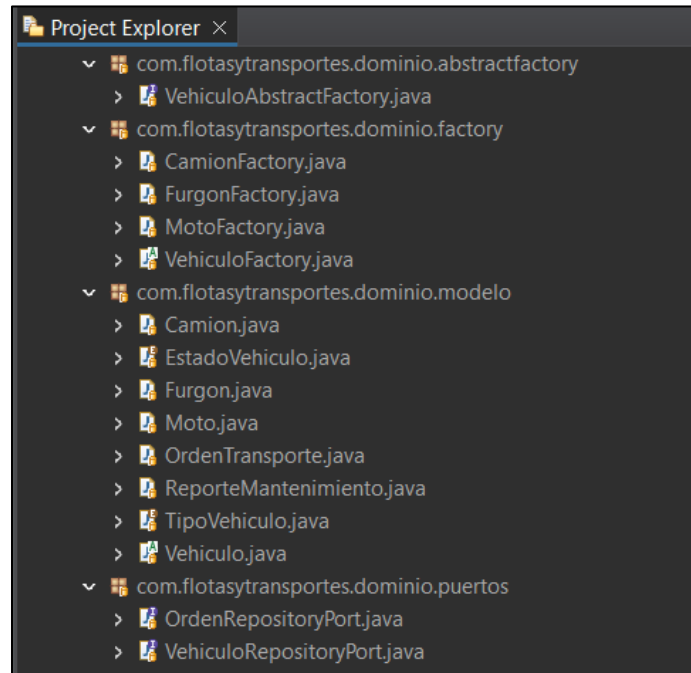
En este modelo, el dominio se ubica en el centro de la arquitectura y representa la parte más estable y crítica del sistema: las reglas del negocio. A diferencia de las arquitecturas tradicionales en capas donde el dominio suele terminar acoplado a tecnologías como frameworks web o librerías de persistencia la arquitectura hexagonal protege la lógica del negocio colocándola en el núcleo y obligando a que todo lo externo dependa de él, y no al revés.

El sistema se organiza alrededor de un núcleo (dominio) y se conecta con el mundo exterior mediante puertos y adaptadores.

- **Puertos:** Son interfaces definidas por el dominio que establecen contratos de comunicación.
 - **Puertos de entrada:** representan los casos de uso que el sistema expone.
 - **Puertos de salida:** representan servicios externos que el dominio necesita (persistencia, servicios externos, etc.).
- **Adaptadores:** Son implementaciones concretas de esos puertos. Permiten que tecnologías específicas (como controladores REST, bases de datos o APIs externas) interactúen con el sistema sin modificar el dominio.

Este enfoque permite que el sistema tenga múltiples puntos de entrada y salida (por ejemplo, API REST, consola, mensajería, base de datos), todos conectados al núcleo mediante contratos bien definidos, sin que el dominio conozca detalles técnicos.

CAPA DE DOMINIO



La capa de dominio constituye el centro arquitectónico del sistema. Está conformada por los siguientes paquetes:

- com.flotasytransportes.dominio.modelo
- com.flotasytransportes.dominio.factory
- com.flotasytransportes.dominio.puertos
- com.flotasytransportes.dominio.abstractfactory

Aquí se concentra la lógica real del negocio logístico.

➤ **MODELO**

En el paquete dominio.modelo se encuentran las entidades principales del sistema, tales como:

- Vehiculo
- Camion
- Furgon

- Moto
- OrdenTransporte
- ReporteMantenimiento
- EstadoVehiculo
- TipoVehiculo

Estas clases representan el modelo conceptual del negocio de flotas. No son simples estructuras de datos, sino que encapsulan comportamiento y reglas propias del dominio, como los estados operativos de un vehículo, la relación entre órdenes y unidades de transporte, o la clasificación por tipo.

Un aspecto fundamental es que estas clases no contienen anotaciones de Spring ni de JPA. Esto demuestra que el dominio se mantiene completamente aislado de la tecnología, cumpliendo el principio central de la arquitectura hexagonal: la lógica de negocio no depende del framework.

➤ **FACTORY**

En el paquete dominio.factory se encuentran clases como:

- VehiculoFactory
- CamionFactory
- FurgonFactory
- MotoFactory

Estas clases encapsulan la lógica de creación de los distintos tipos de vehículos. Desde el punto de vista arquitectónico, esta decisión es relevante porque la creación de entidades forma parte del negocio y no debe delegarse a la infraestructura.

Al centralizar la construcción de objetos dentro del dominio:

- Se evita la dispersión de lógica de instanciación.
- Se mantiene bajo acoplamiento.
- Se facilita la extensión del sistema ante nuevos tipos de vehículos.

➤ PORT

El paquete dominio.puertos contiene interfaces como:

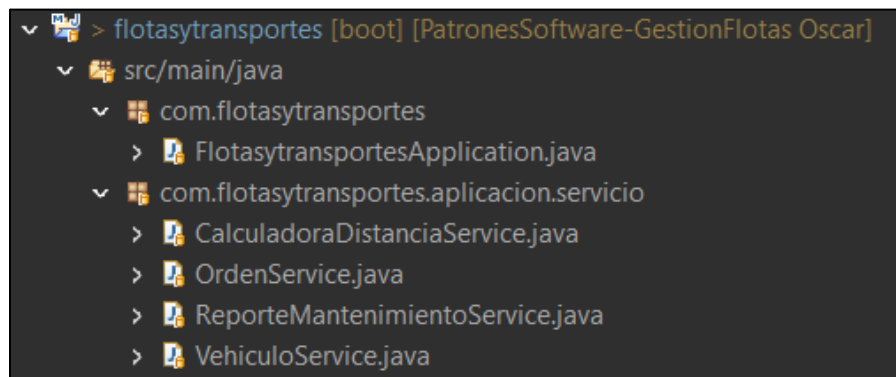
- OrdenRepositoryPort
- VehiculoRepositoryPort

Estos elementos representan los puertos de salida del sistema. Es decir, son contratos definidos por el dominio para interactuar con el exterior, particularmente con la persistencia.

El dominio declara qué necesita (guardar vehículos, consultar órdenes, persistir información), pero no define cómo se implementa. La implementación concreta se delega a la infraestructura.

Este diseño materializa el principio de Inversión de Dependencias, ya que la infraestructura depende de interfaces del dominio, y no al contrario.

CAPA DE APLICACIÓN



La capa de aplicación está representada por el paquete:

- com.flotasytransportes.aplicacion.servicio

Incluye clases como:

- VehiculoService
- OrdenService

- CalculadoraDistanciaService
- ReporteMantenimientoService

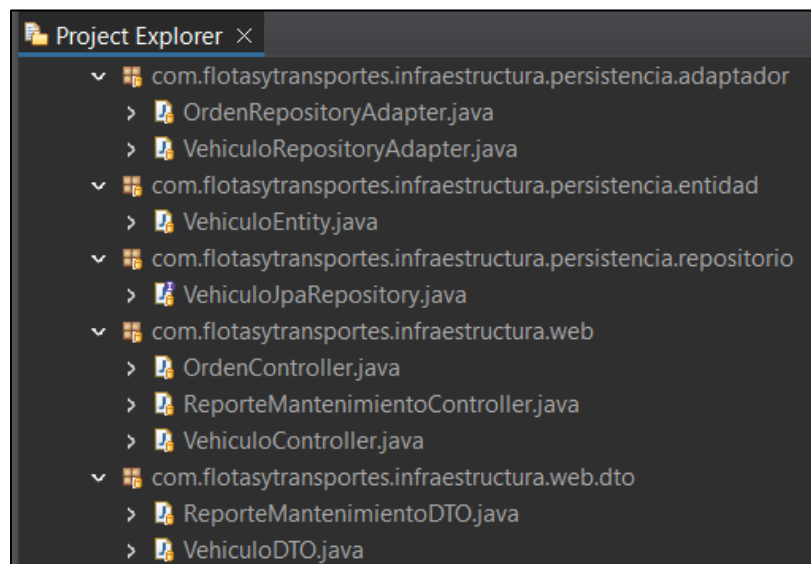
Esta capa no contiene lógica técnica ni detalles de persistencia. Su función es ejecutar los casos de uso del sistema.

Por ejemplo, un servicio puede:

1. Recibir una solicitud proveniente de la capa web.
2. Utilizar una fábrica del dominio para crear una entidad.
3. Invocar un puerto de repositorio para persistir la información.
4. Coordinar el flujo completo del caso de uso.

Es importante destacar que la capa de aplicación no es infraestructura. No maneja directamente la base de datos ni detalles de HTTP. Su responsabilidad es coordinar el dominio, manteniendo la coherencia del flujo operativo.

CAPA DE INFRAESTRUCTURA



La capa de infraestructura contiene las implementaciones técnicas que permiten conectar el dominio con el entorno externo.

➤ **ADAPTADORES**

En el paquete `infrastructure.persistencia.adaptador` se encuentran:

- `OrdenRepositoryAdapter`
- `VehiculoRepositoryAdapter`

Estas clases implementan las interfaces definidas en `dominio.puertos`. Aquí se concreta la comunicación con la base de datos.

➤ **ENTIDADES**

El paquete `infrastructure.persistencia.entidad` contiene clases como:

- `VehiculoEntity`

Es importante señalar que estas entidades son distintas a las entidades del dominio. Esto evita que el modelo del negocio quede contaminado con anotaciones técnicas, manteniendo una separación limpia entre modelo conceptual y modelo de base de datos.

➤ **REPOSITORIOS**

En `infrastructure.persistencia.repositorio` se encuentra:

- `VehiculoJpaRepository`

Aquí aparece el uso de tecnologías como Spring Data JPA y Spring Boot. Estas dependencias están correctamente ubicadas en la capa externa, cumpliendo el principio de independencia tecnológica del dominio.

PATRONES DE DISEÑO DE SOFTWARE

PATRON SINGLETON

El Singleton es un patrón de diseño creacional cuyo objetivo es garantizar que una clase tenga una única instancia durante todo el ciclo de vida de la aplicación y proporcionar un punto de acceso global controlado a dicha instancia.

El patrón se utiliza cuando:

- No tiene sentido crear múltiples instancias del mismo objeto.
- El componente coordina lógica central del sistema.
- Se desea controlar el acceso a recursos compartidos.

Tradicionalmente, en Java, se implementa mediante:

- Constructor privado.
- Atributo estático.
- Método público getInstance().

Sin embargo, en aplicaciones modernas con contenedores IoC, como **SPRING**, el patrón puede implementarse de forma gestionada por el framework.

En este proyecto, el patrón Singleton no fue implementado manualmente, sino que se utiliza el Singleton gestionado por el contenedor de Spring.

En Spring Framework, el alcance por defecto de los componentes es patrón **SINGLETON**.

Esto significa que:

- Se crea una única instancia.
- Se reutiliza en toda la aplicación.
- No se permite la creación manual repetida.



IMPORTANTE

IMPLEMENTACION EN EL CODIGO

El patrón Singleton se evidencia en las clases anotadas con la siguiente sentencia:

- @Service
- @Repository
- @Component

```

CalculadoraDistanciaService.java ×
1 package com.flotasytransportes.aplicacion.servicio;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class CalculadoraDistanciaService {
7
8     private static final double RADIO_TIERRA_KM = 6371;
9
10    public double calcularDistancia(double lat1, double lon1, double lat2, double lon2) {
11
12        double dLat = Math.toRadians(lat2 - lat1);
13        double dLon = Math.toRadians(lon2 - lon1);
14
15        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) + Math.cos(Math.toRadians(lat1))
16            * Math.cos(Math.toRadians(lat2)) * Math.sin(dLon / 2) * Math.sin(dLon / 2);
17
18        double c = 2 * Math.asin(Math.sqrt(a));
19
20        return RADIO_TIERRA_KM * c;
21    }
22 }
23

```

```

OrdenRepositoryAdapter.java ×
1 package com.flotasytransportes.infraestructura.persistencia.adaptador;
2
3 import org.springframework.stereotype.Repository;
4
5 @Repository
6 public class OrdenRepositoryAdapter implements OrdenRepositoryPort {
7
8     @Override
9     public OrdenTransporte guardar(OrdenTransporte orden) {
10        // lógica de guardado
11        return orden;
12    }
13 }
14
15

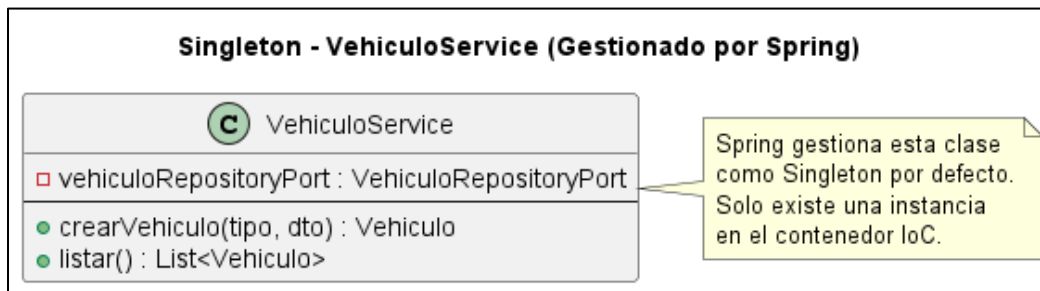
```

El patrón Singleton se utiliza en tu Sistema de Gestión de Flotas porque:

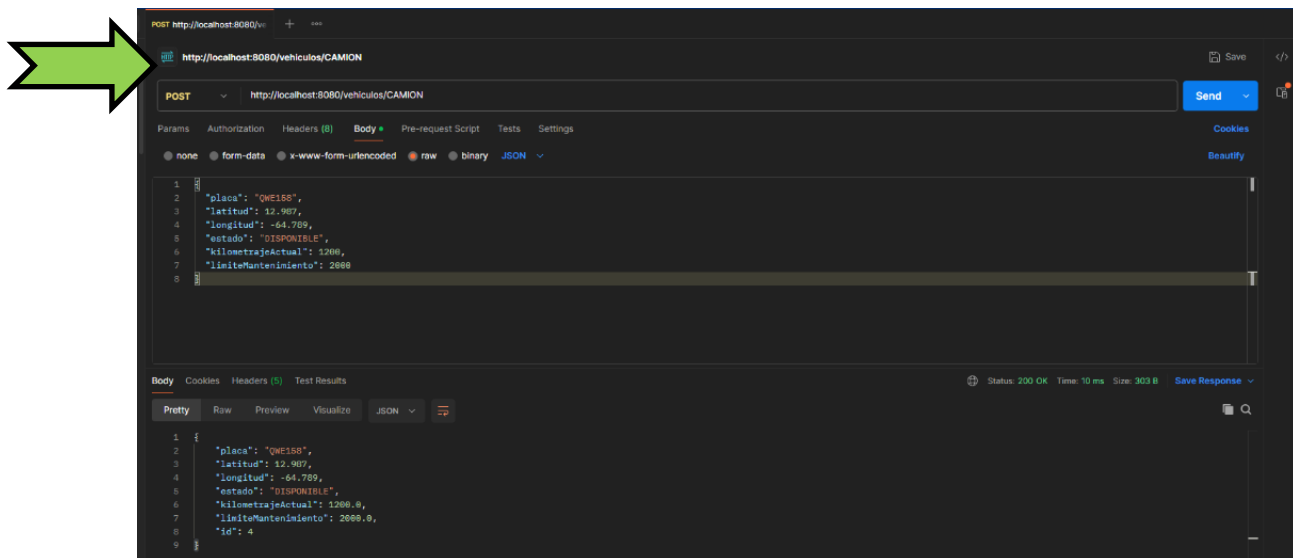
- Los servicios representan coordinadores centrales del negocio.
- Los repositorios gestionan recursos compartidos.
- No se requiere estado independiente por instancia.
- Se busca eficiencia y coherencia global.
- El framework utilizado lo implementa de manera natural y segura.

En este contexto, el Singleton no es una elección arbitraria, sino una consecuencia lógica del diseño del sistema y de la naturaleza de sus componentes.

DIAGRAMA UML



PRUEBAS



La imagen anterior muestra la ejecución de una petición HTTP al endpoint expuesto por VehiculoController, el cual utiliza internamente la clase VehiculoService, anotada con `@Service`.

Durante la ejecución de múltiples solicitudes desde Postman, se imprimió en consola el hashCode de la instancia de VehiculoService. Se evidenció que el identificador de la instancia permanece constante en cada petición, lo que confirma que el contenedor de Spring crea una única instancia de la clase y la reutiliza durante todo el ciclo de vida de la aplicación.

Este comportamiento demuestra la aplicación del patrón Singleton en la capa de servicios, ya que existe una sola instancia compartida que coordina la lógica de negocio relacionada con la gestión de vehículos.

PATRÓN FACTORY METHOD

El Factory Method es un patrón de diseño creacional cuyo propósito es definir una interfaz para la creación de objetos, permitiendo que las subclases decidan qué clase concreta instanciar.

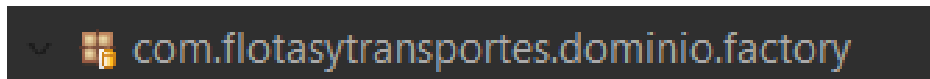
En lugar de crear objetos directamente mediante *new ClaseConcreta()* la creación se delega a una fábrica que encapsula la lógica de instanciación.

Este patrón se utiliza cuando:

- Existen múltiples variantes de un mismo tipo de objeto.
- La creación depende de una condición.
- Se desea reducir el acoplamiento entre el cliente y las clases concretas.

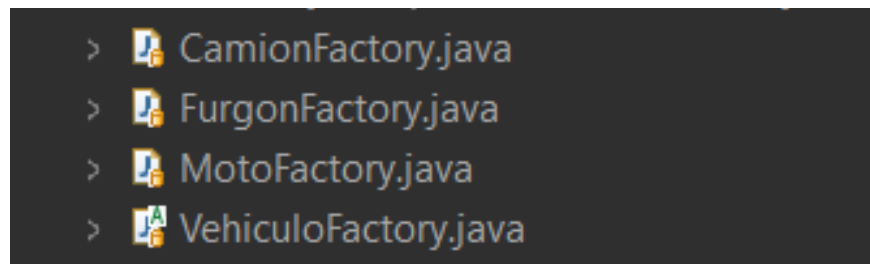
IMPLEMENTACION EN EL CODIGO

En este proyecto, el patrón Factory Method se encuentra en el paquete:



```
com.flotasytransportes.dominio.factory
```

Clases involucradas:



```
> CamionFactory.java  
> FurgonFactory.java  
> MotoFactory.java  
> VehiculoFactory.java
```

Estas clases encapsulan la lógica de creación de los distintos tipos de vehículos.

En el sistema existen distintos tipos de vehículos:

- Camión
- Furgón
- Moto

Cada uno puede tener características o comportamientos particulares.

Si la creación se hiciera directamente en el servicio con múltiples condicionales:

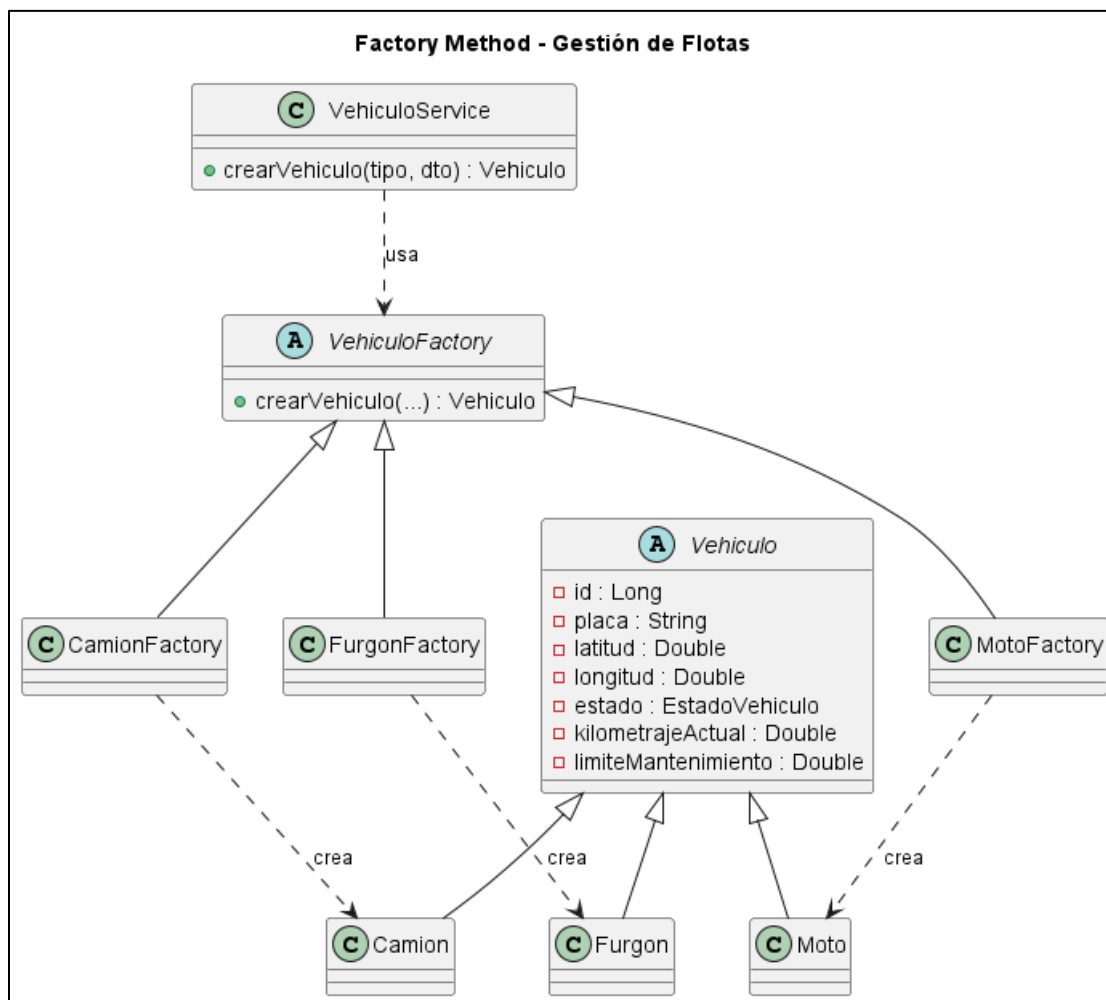
```
if(tipo.equals("CAMION")) {  
    return new Camion(...);  
} else if(tipo.equals("FURGON")) {  
    return new Furgon(...);  
}
```

Esto generaría:

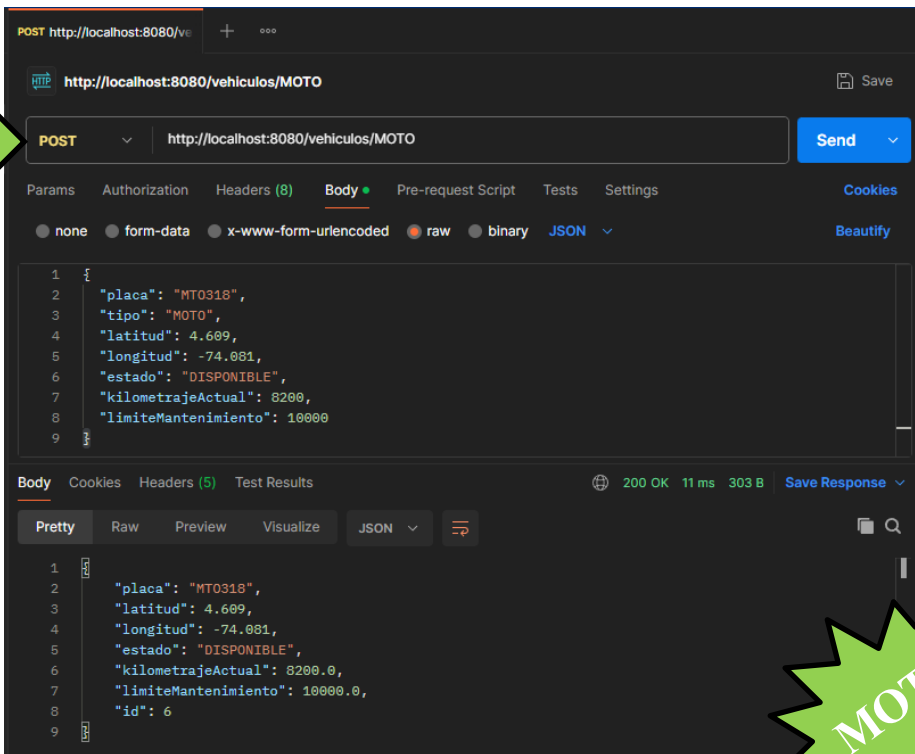
- Alto acoplamiento
- Violación del principio Open/Closed
- Código difícil de mantener
- Lógica de creación dispersa

El Factory Method centraliza esa responsabilidad.

DIAGRAMA UML



PRUEBAS



POST http://localhost:8080/vehiculos/MOTO

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

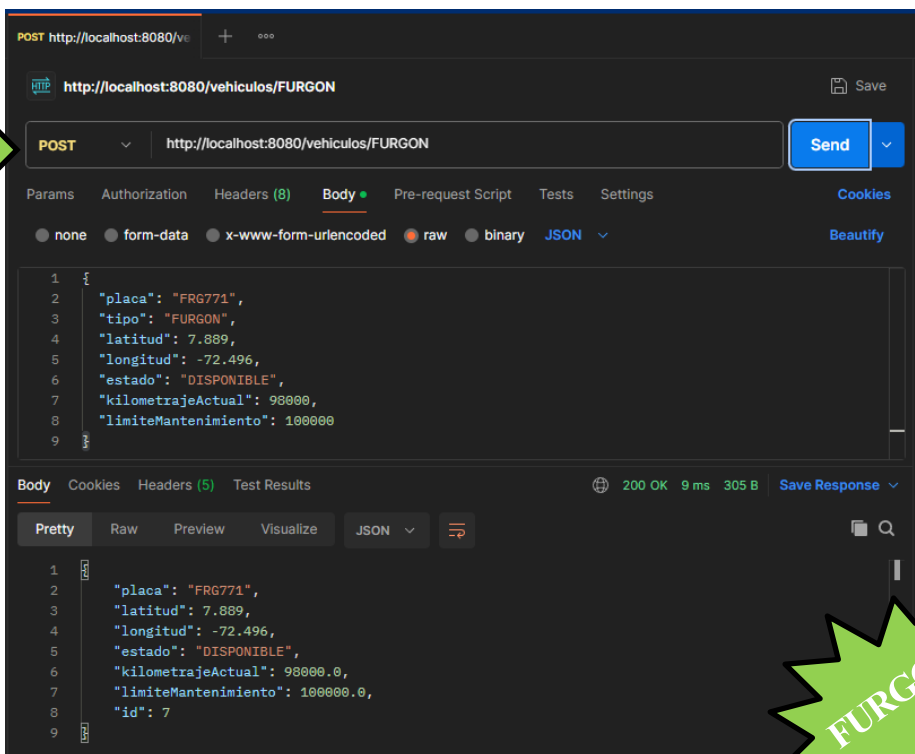
```
1 {
2   "placa": "MT0318",
3   "tipo": "MOTO",
4   "latitud": 4.609,
5   "longitud": -74.081,
6   "estado": "DISPONIBLE",
7   "kilometrajeActual": 8200,
8   "limiteMantenimiento": 10000
9 }
```

Body Cookies Headers (5) Test Results 200 OK 11 ms 303 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "placa": "MT0318",
3   "latitud": 4.609,
4   "longitud": -74.081,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 8200.0,
7   "limiteMantenimiento": 10000.0,
8   "id": 6
9 }
```

MOTO



POST http://localhost:8080/vehiculos/FURGON

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

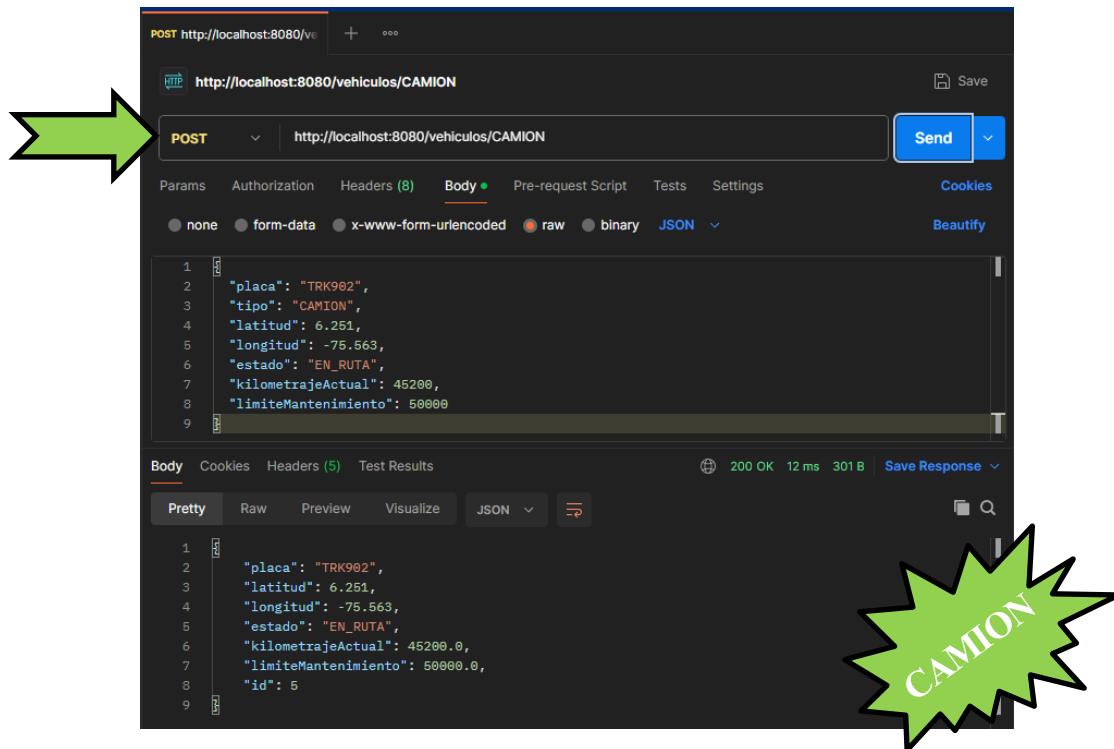
```
1 {
2   "placa": "FRG771",
3   "tipo": "FURGON",
4   "latitud": 7.889,
5   "longitud": -72.496,
6   "estado": "DISPONIBLE",
7   "kilometrajeActual": 98000,
8   "limiteMantenimiento": 100000
9 }
```

Body Cookies Headers (5) Test Results 200 OK 9 ms 305 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "placa": "FRG771",
3   "latitud": 7.889,
4   "longitud": -72.496,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 98000.0,
7   "limiteMantenimiento": 100000.0,
8   "id": 7
9 }
```

FURGON



Con el fin de validar la correcta implementación del patrón Factory Method dentro del sistema de gestión de flotas, se realizaron pruebas de creación de distintos tipos de vehículos mediante solicitudes enviadas al endpoint correspondiente del módulo de vehículos.

Durante las pruebas se enviaron peticiones desde POSTMAN para registrar vehículos de diferentes tipos, específicamente Moto, Furgón y Camión. Cada solicitud incluye la información necesaria del vehículo, como la placa, tipo de vehículo y demás atributos definidos en el DTO utilizado por el sistema.

Cuando el sistema recibe la solicitud, la clase VehiculoService delega la creación del objeto a la fábrica correspondiente según el tipo de vehículo indicado. De esta manera, el sistema selecciona automáticamente la clase de fábrica adecuada (MotoFactory, FurgonFactory o CamionFactory), la cual se encarga de construir la instancia concreta del vehículo.

Los resultados observados en las respuestas del sistema y en los registros almacenados evidencian que cada solicitud genera correctamente el tipo de objeto esperado dentro del dominio. Esto confirma que la lógica de creación de objetos no se encuentra acoplada directamente al servicio, sino que está centralizada en las fábricas especializadas.

PATRÓN ABSTRACT FACTORY

Es un patrón de diseño creacional cuyo propósito es proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Este patrón permite que el código cliente utilice diferentes conjuntos de objetos compatibles entre sí sin quedar acoplado a sus implementaciones específicas.

La idea principal del patrón es separar la lógica de creación de objetos de su uso, delegando dicha responsabilidad a una fábrica abstracta que define los métodos de creación, mientras que las fábricas concretas implementan esos métodos para instanciar los productos específicos. Este patrón es especialmente útil cuando un sistema necesita trabajar con múltiples familias de objetos que deben utilizarse conjuntamente, garantizando que los objetos creados sean compatibles entre sí.

Uno de los aspectos importantes del **ABSTRACT FACTORY** es que generalmente utiliza internamente el patrón Factory Method para realizar la instanciación de los productos concretos. Mientras que el **FACTORY METHOD** se enfoca en la creación de un solo producto, el **ABSTRACT FACTORY** coordina la creación de conjuntos completos de objetos relacionados. Entre las principales ventajas de este patrón se encuentran el desacoplamiento entre el cliente y las clases concretas, la facilidad para cambiar o ampliar familias de productos y el cumplimiento del principio Open/Closed, ya que es posible agregar nuevas familias de objetos sin modificar el código existente.

Sin embargo, también presenta algunas desventajas, como el aumento en el número de clases dentro del sistema y la dificultad para agregar nuevos tipos de productos si no fueron contemplados inicialmente en la interfaz de la fábrica. En sistemas complejos o escalables, el patrón **ABSTRACT FACTORY** resulta especialmente útil para mantener una arquitectura organizada, flexible y extensible, permitiendo que la creación de objetos se gestione de manera centralizada y coherente con el dominio del sistema.

IMPLEMENTACION EN EL CODIGO

En el proyecto se implementa el patrón **ABSTRACT FACTORY** para gestionar la creación de los distintos tipos de vehículos dentro de la flota caracterizados por su tipo de energía.

La clase abstracta definida en el dominio es:

```
VehiculoAbstractFactory.java ×
1 package com.flotasytransportes.dominio.abstractfactory;
2
3 import com.flotasytransportes.dominio.modelo.Vehiculo;
4
5
6 public interface VehiculoAbstractFactory {
7
8
9     Vehiculo crearVehiculo(String placa, Double latitud, Double longitud, EstadoVehiculo estado, TipoEnergia tipoEnergia,
10         Double kilometrajeActual, Double limiteMantenimiento);
11
12 }
```

Esta interfaz establece el contrato que deben cumplir todas las clases encargadas de crear vehículos dentro del sistema.

A partir de esta clase abstracta se implementan las siguientes clases concretas:

- CamionFactory
- FurgonFactory
- MotoFactory

Cada una de estas clases se encarga de crear un tipo específico de vehículo.

```
VehiculoFactory.java ×
1 package com.flotasytransportes.dominio.factory;
2
3 import com.flotasytransportes.dominio.abstractfactory.VehiculoAbstractFactory;
4
5
6 public abstract class VehiculoFactory implements VehiculoAbstractFactory {
7
8     // ESTE ES EL FACTORY METHOD
9     public abstract Vehiculo crearVehiculo(
10         String placa,
11         Double latitud,
12         Double longitud,
13         EstadoVehiculo estado,
14         TipoEnergia tipoEnergia,
15         Double kilometrajeActual,
16         Double limiteMantenimiento
17     );
18 }
```

En esta estructura se observa la aplicación del patrón **ABSTRACT FACTORY** para organizar la creación de los distintos tipos de vehículos del sistema. La interfaz VehiculoAbstractFactory actúa como la clase abstracta, ya que define el método crearVehiculo () que deben implementar todas las fábricas encargadas de crear objetos del dominio.

A partir de esta abstracción se implementan las clases concretas, como CamionFactory, FurgonFactory y MotoFactory. Cada una de estas clases se especializa en la creación de un tipo específico de vehículo según su tipo de energía.

Los objetos que finalmente se crean (Camión, Furgon y Moto) corresponden a los productos concretos del patrón. Todos ellos pertenecen a la misma familia de objetos del dominio (Vehículo), pero representan diferentes tipos de vehículos dentro del sistema de gestión de flotas dado que algunos su tipo de energía es ELECTRICA o mediante el gasto de GASOLINA.

```
VehiculoService.java x
60
61● public Vehiculo crearVehiculo(TipoVehiculo tipo, VehiculoDTO dto) {
62
63     VehiculoAbstractFactory abstractfactory;
64
65     switch (tipo) {
66         case CAMION:
67             abstractfactory = new CamionFactory();
68             break;
69         case MOTO:
70             abstractfactory = new MotoFactory();
71             break;
72         case FURGON:
73             abstractfactory = new FurgonFactory();
74             break;
75         default:
76             throw new IllegalArgumentException("Tipo no soportado");
77     }
78
79     Vehiculo vehiculo = abstractfactory.crearVehiculo(
80         dto.getPlaca(),
81         dto.getLatitud(),
82         dto.getLongitud(),
83         dto.getEstado(),
84         dto.getTipoEnergia(),
85         dto.getKilometrajeActual(),
86         dto.getLimiteMantenimiento()
87     );
88
89     return vehiculoRepository.guardar(vehiculo);
90 }
91
92 }
```

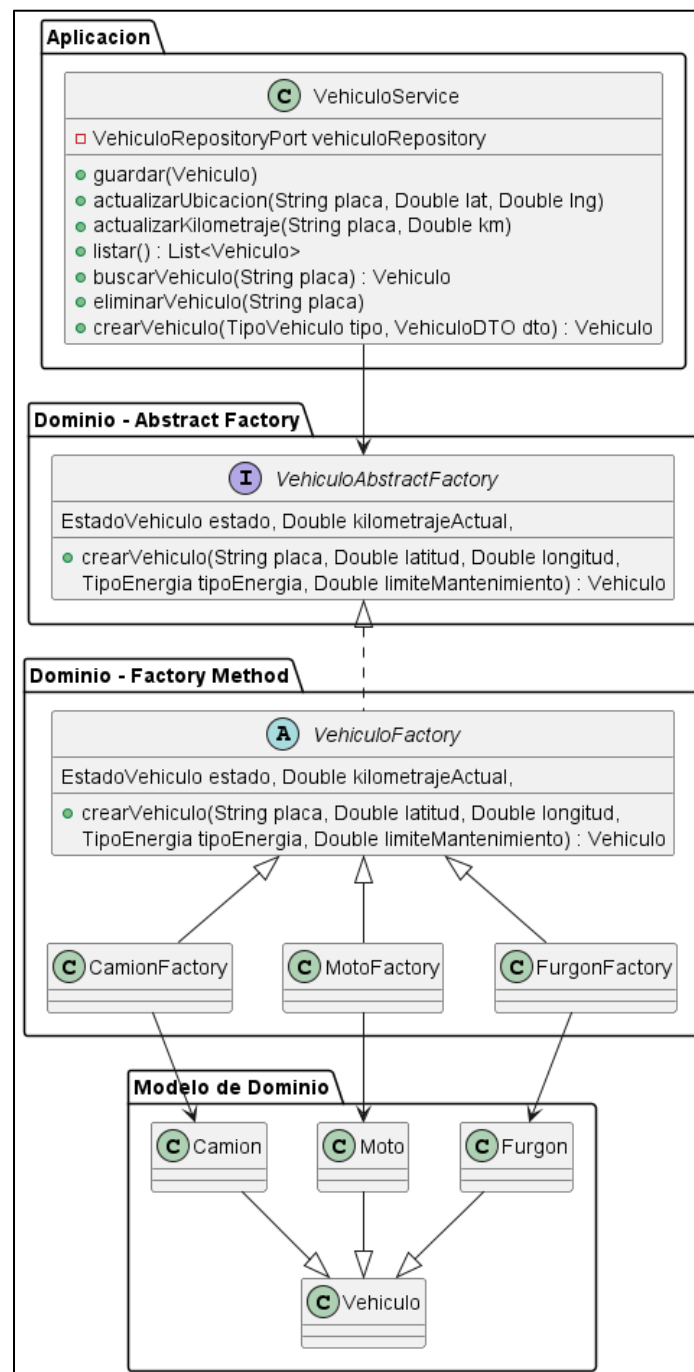
Dentro de la clase VehiculoService, este método utiliza el patrón ABSTRACT FACTORY para crear diferentes tipos de vehículos de manera desacoplada.

Primero, se declara una variable de tipo VehiculoAbstractFactory, que representa la clase abstracta encargada de definir cómo se crean los vehículos. Luego, mediante una estructura switch, el sistema selecciona la clase concreta correspondiente según el tipo de vehículo solicitado (CamionFactory, MotoFactory o FurgonFactory).

Una vez elegida la clase, se llama al método crearVehiculo () pasando los datos del VehiculoDTO. Cada clase concreta se encarga de crear el objeto específico (Camion, Moto o Furgon).

De esta forma, el servicio no depende directamente de las clases concretas, sino de la abstracción de la clase, lo que permite mantener el sistema desacoplado y facilita la incorporación de nuevos tipos de vehículos sin modificar la lógica principal.

DIAGRAMA UML

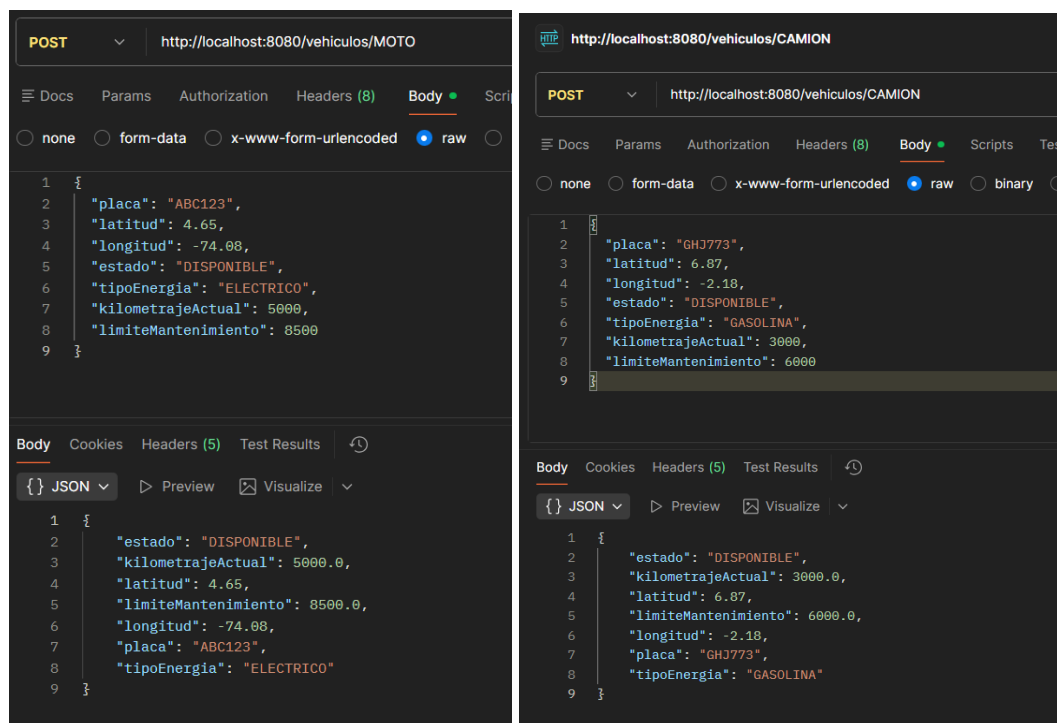


PRUEBAS

Esta sección presenta las pruebas integrales del módulo de vehículos, en el cual se implementa el patrón **ABSTRACT FACTORY** para la creación de distintos tipos de vehículos dentro del sistema. Las pruebas validan el correcto funcionamiento de las operaciones **CRUD** (Crear, Leer, Actualizar y Eliminar) utilizando el endpoint /vehículos, permitiendo gestionar vehículos a través de su placa como identificador principal.

POST / PUT	vehiculos/{placa}	Creación de un vehiculo nuevo / Actualización del vehiculo
GET	vehiculos	Lista completa de vehiculos
GET	vehiculos/{placa}	Buscar un vehiculo
DELETE	vehiculos/{placa}	Eliminar un vehiculo existente

CREACIÓN DE UN VEHICULO NUEVO



The image displays two screenshots of a REST client interface, likely Postman, showing the creation of a new vehicle via a POST request.

Left Screenshot (MOTO):

- Method: POST
- URL: `http://localhost:8080/vehiculos/MOTO`
- Body Type: raw
- Body (JSON):

```
{  "placa": "ABC123",  "latitud": 4.65,  "longitud": -74.08,  "estado": "DISPONIBLE",  "tipoEnergia": "ELECTRICO",  "kilometrajeActual": 5000,  "limiteMantenimiento": 8500}
```
- Body Tab: JSON
- Body (JSON):

```
{  "estado": "DISPONIBLE",  "kilometrajeActual": 5000.0,  "latitud": 4.65,  "limiteMantenimiento": 8500.0,  "longitud": -74.08,  "placa": "ABC123",  "tipoEnergia": "ELECTRICO"}
```

Right Screenshot (CAMION):

- Method: POST
- URL: `http://localhost:8080/vehiculos/CAMION`
- Body Type: raw
- Body (JSON):

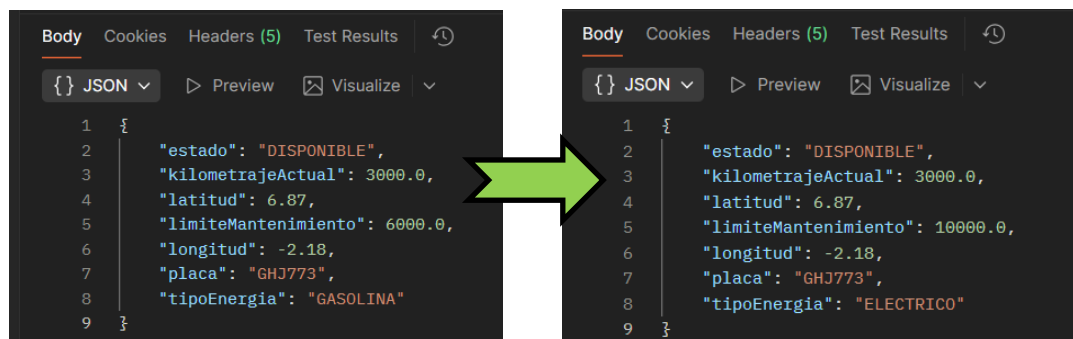
```
{  "placa": "GHJ773",  "latitud": 6.87,  "longitud": -2.18,  "estado": "DISPONIBLE",  "tipoEnergia": "GASOLINA",  "kilometrajeActual": 3000,  "limiteMantenimiento": 6000}
```
- Body Tab: JSON
- Body (JSON):

```
{  "estado": "DISPONIBLE",  "kilometrajeActual": 3000.0,  "latitud": 6.87,  "limiteMantenimiento": 6000.0,  "longitud": -2.18,  "placa": "GHJ773",  "tipoEnergia": "GASOLINA"}
```

Se envió una solicitud POST al endpoint /vehiculos, proporcionando los datos necesarios para registrar un nuevo vehículo, incluyendo la placa, el tipo de vehículo y demás atributos requeridos. Dependiendo del tipo de energía indicado en la solicitud, el sistema utiliza el

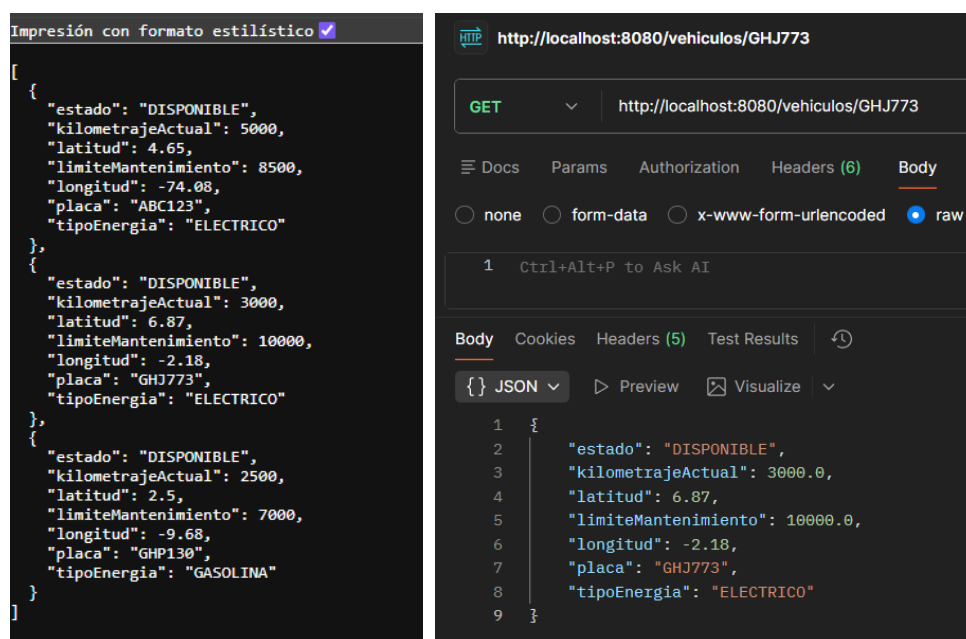
patrón Abstract Factory para seleccionar automáticamente la fábrica correspondiente, como CamionFactory, MotoFactory o FurgonFactory, encargada de crear la instancia concreta del vehículo ya sea ELECTRICO o a GASOLINA.

ACTUALIZACION DEL VEHICULO



Actualización de vehículos mediante solicitudes PUT al endpoint /vehiculos/{placa}. Estas pruebas permitieron verificar que el sistema puede modificar correctamente la información de un vehículo existente manteniendo la coherencia de los datos dentro del sistema.

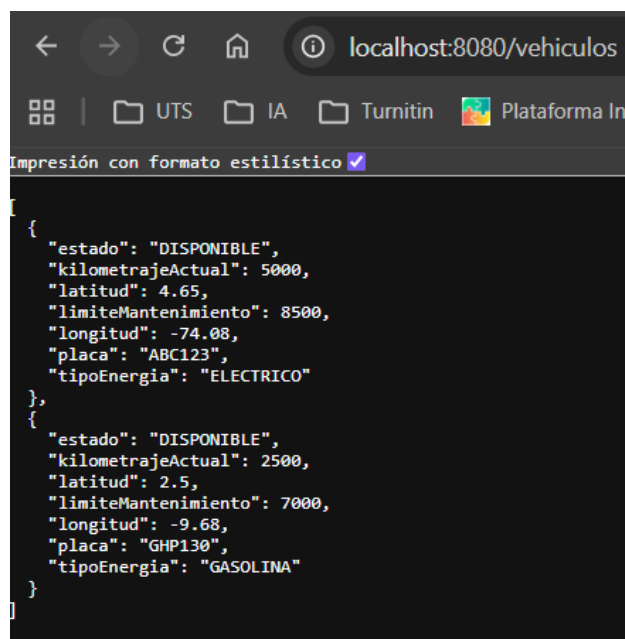
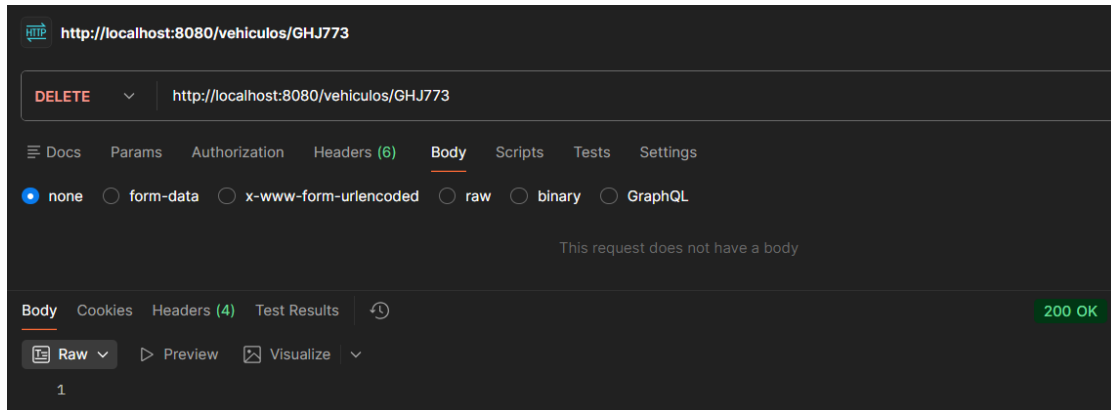
LISTA COMPLETA DE VEHICULOS



Utilizando solicitudes GET para obtener la lista completa de vehículos registrados y para buscar un vehículo específico por su placa. Los resultados mostraron que el sistema devuelve

correctamente la información almacenada, confirmando que las entidades creadas mediante las fábricas concretas se integran correctamente con la lógica del sistema.

ELIMINAR UN VEHICULO EXISTENTE



Finalmente, se realizaron pruebas de eliminación de vehículos mediante solicitudes DELETE al endpoint /vehiculos/{placa}. Estas pruebas confirmaron que el sistema puede eliminar correctamente un vehículo existente de la lista registrada.

Los resultados obtenidos en todas estas pruebas demuestran que el patrón Abstract Factory está funcionando correctamente dentro del sistema, permitiendo crear diferentes tipos de vehículos de forma desacoplada y manteniendo una arquitectura flexible y extensible.

PATRON BUILDER

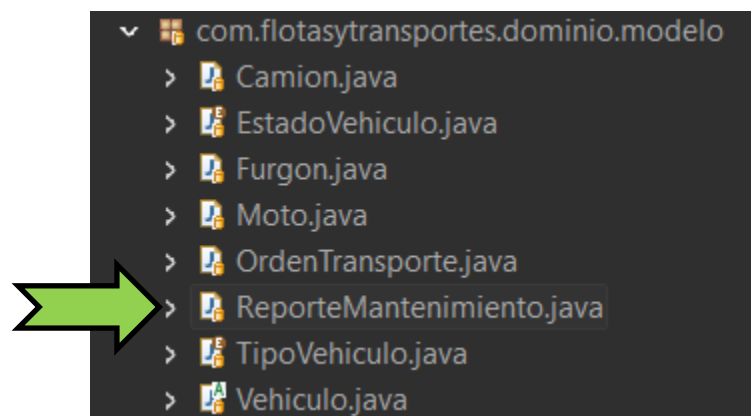
Es un patrón creacional cuyo propósito principal es construir objetos complejos paso a paso, separando el proceso de construcción de la representación final del objeto. Este patrón permite que un mismo proceso de construcción pueda crear diferentes representaciones de un objeto dependiendo de los parámetros o configuraciones utilizadas durante su creación.

En el desarrollo de software orientado a objetos, es común encontrar clases que requieren una gran cantidad de atributos para representar correctamente una entidad del sistema. Cuando una clase posee muchos atributos, especialmente cuando algunos son obligatorios y otros opcionales, el uso de constructores tradicionales puede generar código difícil de leer, mantener o extender. En estos casos, el patrón Builder proporciona una solución estructurada que permite crear objetos de manera progresiva y controlada.

El patrón Builder propone la creación de una clase constructora que se encarga de configurar los diferentes atributos del objeto final. En lugar de instanciar directamente el objeto mediante un constructor con múltiples parámetros, el desarrollador utiliza el Builder para establecer cada atributo mediante métodos específicos, y finalmente invoca un método el cual se encarga de generar la instancia completa del objeto.

IMPLEMENTACION EN EL CODIGO

El patrón Builder se implementa en la clase:



Esta clase representa un reporte de mantenimiento de un vehículo, el cual contiene múltiples atributos relacionados con el mantenimiento realizado.

```

ReporteMantenimiento.java X
38
40
41 public static class Builder {
42     // OBLIGATORIOS
43     private String placaVehiculo;
44     private Double kilometraje;
45     private String fecha;
46     private String tipoMantenimiento;
47
48     // OPCIONALES
49     private String observaciones;
50     private String prioridad;
51     private String tecnicoResponsable;
52     private String taller;
53     private Double costoEstimado;
54     private Integer tiempoEstimadoHoras;
55     private Boolean requiereRepuestos;

```

```

ReporteMantenimiento.java X
56 // CONSTRUCTOR CON OBLIGATORIOS
57 public Builder(String placaVehiculo, Double kilometraje, String fecha, String tipoMantenimiento) {
58     this.placaVehiculo = placaVehiculo;
59     this.kilometraje = kilometraje;
60     this.fecha = fecha;
61     this.tipoMantenimiento = tipoMantenimiento;
62 }
63
64
65
66 public Builder observaciones(String observaciones) {
67     this.observaciones = observaciones;
68     return this;
69 }
70
71 public Builder prioridad(String prioridad) {
72     this.prioridad = prioridad;
73     return this;
74 }
75
76 public Builder tecnicoResponsable(String tecnicoResponsable) {
77     this.tecnicoResponsable = tecnicoResponsable;
78     return this;
79 }
80
81 public Builder taller(String taller) {
82     this.taller = taller;
83     return this;
84 }
85
86 public Builder costoEstimado(Double costoEstimado) {
87     this.costoEstimado = costoEstimado;
88     return this;
89 }

```

La clase Builder es una clase interna estática dentro de ReporteMantenimiento cuya función es implementar el patrón de diseño Builder. Este patrón permite construir objetos complejos paso a paso, separando los atributos obligatorios de los opcionales. Gracias a esta estructura, el proceso de creación del objeto se vuelve más organizado, legible y flexible dentro del sistema.

Dentro del Builder se definen primero los atributos obligatorios del reporte de mantenimiento, los cuales son placaVehiculo, kilometraje, fecha y tipoMantenimiento. Estos datos son necesarios para que el sistema pueda identificar correctamente el vehículo y el tipo

de mantenimiento que se realizará. Por esta razón, estos atributos se reciben a través del constructor del Builder, garantizando que siempre estén presentes cuando se cree un reporte.

Además de los campos obligatorios, el Builder también incluye varios atributos opcionales, como observaciones, prioridad, técnico responsable, taller, costo estimado, tiempo estimado de trabajo, si requiere repuestos, nivel de desgaste y el kilometraje recomendado para el próximo mantenimiento. Estos atributos permiten agregar información adicional al reporte, pero no son indispensables para su creación.

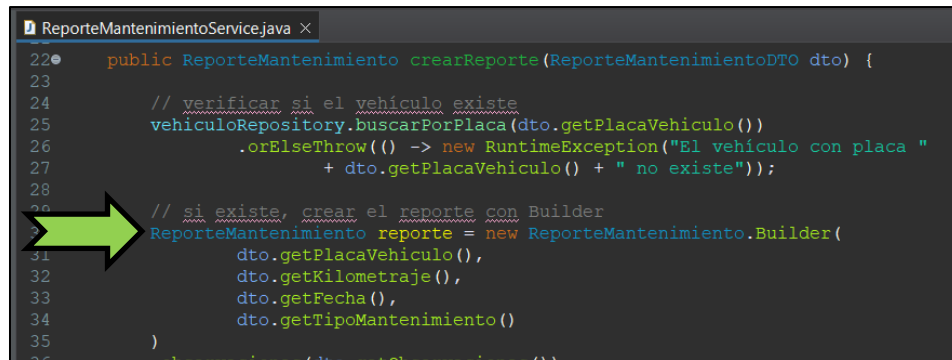


```
ReporteMantenimientoController.java x
1 package com.flotasytransportes.infraestructura.web;
2
3 import com.flotasytransportes.aplicacion.servicio.ReporteMantenimientoService;
4
5 @RestController
6 @RequestMapping("/reportes")
7 public class ReporteMantenimientoController {
8
9     private final ReporteMantenimientoService service;
10
11     public ReporteMantenimientoController(ReporteMantenimientoService service) {
12         this.service = service;
13     }
14
15     @PostMapping("/mantenimiento")
16     public ReporteMantenimiento crear(@Valid @RequestBody ReporteMantenimientoDTO dto) {
17         return service.crearReporte(dto);
18     }
19
20     @GetMapping
21     public List<ReporteMantenimiento> listar() {
22         return service.listarReportes();
23     }
24 }
25
26
27
28
29
30
31 }
```

En la clase ReporteMantenimientoController la implementación del patrón Builder no se realiza directamente, pero esta clase participa en su uso dentro del flujo de la aplicación. El controlador recibe los datos necesarios para crear un reporte de mantenimiento mediante una solicitud HTTP y los envía a la capa de servicio, donde posteriormente se utiliza el patrón Builder para construir el objeto ReporteMantenimiento.

La sección donde se relaciona con la implementación del patrón Builder es el método crear(), anotado con @PostMapping("/mantenimiento"). En este método se recibe un objeto ReporteMantenimientoDTO a través del cuerpo de la petición (@RequestBody). Este DTO contiene los datos que el usuario envía para crear el reporte de mantenimiento. Una vez recibido y validado con @Valid, el controlador envía este objeto al método crearReporte() del servicio ReporteMantenimientoService.

Es precisamente dentro del servicio donde se utiliza el Builder para construir la instancia de ReporteMantenimiento utilizando los datos del DTO. Por lo tanto, el controlador cumple el rol de punto de entrada de los datos que posteriormente serán utilizados por el Builder para crear el objeto final.



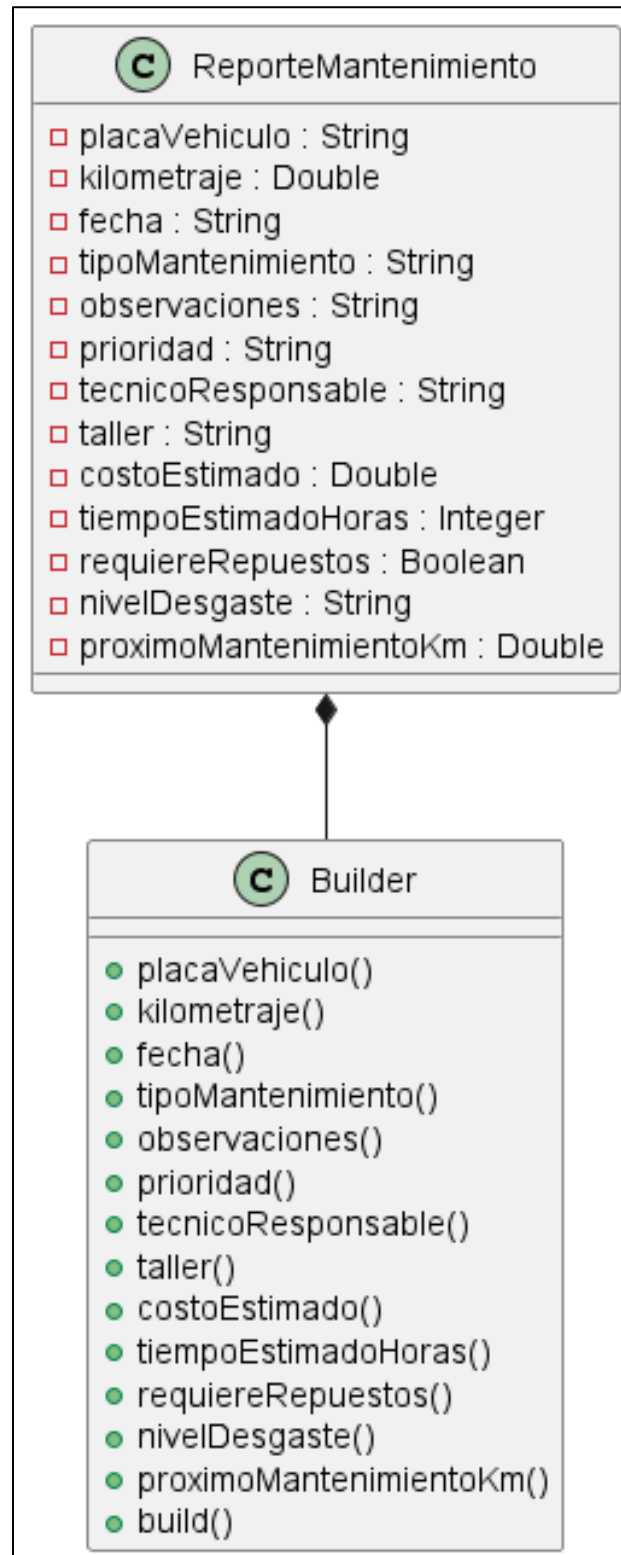
```
ReporteMantenimientoService.java x
22 public ReporteMantenimiento crearReporte(ReporteMantenimientoDTO dto) {
23
24     // verificar si el vehículo existe
25     vehiculoRepository.buscarPorPlaca(dto.getPlacaVehiculo())
26     .orElseThrow(() -> new RuntimeException("El vehículo con placa "
27         + dto.getPlacaVehiculo() + " no existe"));
28
29     // si existe, crear el reporte con Builder
30     ReporteMantenimiento reporte = new ReporteMantenimiento.Builder(
31         dto.getPlacaVehiculo(),
32         dto.getKilometraje(),
33         dto.getFecha(),
34         dto.getTipoMantenimiento()
35     )
36 }
```

En la clase ReporteMantenimientoService se utiliza directamente el patrón Builder dentro del método crearReporte(ReporteMantenimientoDTO dto). En este método primero se verifica si el vehículo existe en el sistema utilizando el repositorio VehiculoRepositoryPort. Si el vehículo no existe, se lanza una excepción para evitar crear un reporte con una placa inválida.

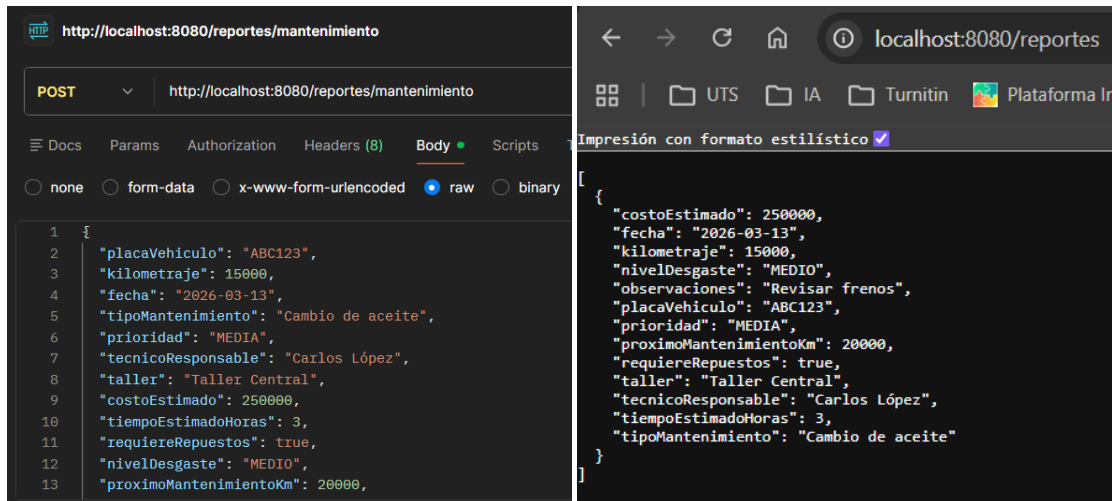
Una vez validado el vehículo, se procede a construir el objeto ReporteMantenimiento utilizando el patrón Builder mediante la instrucción new ReporteMantenimiento.Builder(...). Allí se pasan los datos obligatorios como la placa del vehículo, el kilometraje, la fecha y el tipo de mantenimiento. Luego, mediante métodos encadenados se asignan los atributos opcionales del reporte.

Finalmente, al llamar al método build() se crea el objeto ReporteMantenimiento completo, el cual se guarda en la lista de reportes y se retorna como resultado del método. De esta manera, el patrón Builder permite crear el objeto de forma más clara y organizada.

DIAGRAMA UML



PRUEBAS



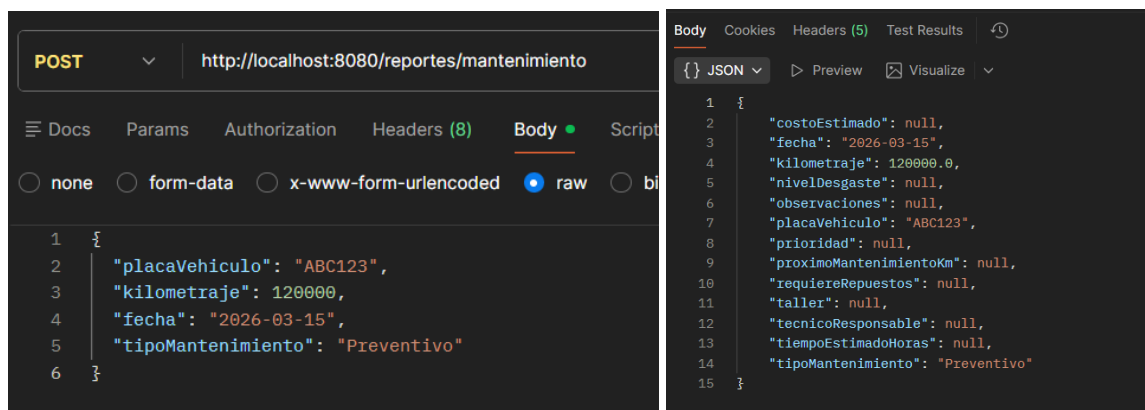
```
POST http://localhost:8080/reportes/mantenimiento

{
  "placaVehiculo": "ABC123",
  "kilometraje": 15000,
  "fecha": "2026-03-13",
  "tipoMantenimiento": "Cambio de aceite",
  "prioridad": "MEDIA",
  "tecnicoResponsable": "Carlos López",
  "taller": "Taller Central",
  "costoEstimado": 250000,
  "tiempoEstimadoHoras": 3,
  "requiereRepuestos": true,
  "nivelDesgaste": "MEDIO",
  "proximoMantenimientoKm": 20000
}
```

```
Impresión con formato estilístico

{
  "costoEstimado": 250000,
  "fecha": "2026-03-13",
  "kilometraje": 15000,
  "nivelDesgaste": "MEDIO",
  "observaciones": "Revisar frenos",
  "placaVehiculo": "ABC123",
  "prioridad": "MEDIA",
  "proximoMantenimientoKm": 20000,
  "requiereRepuestos": true,
  "taller": "Taller Central",
  "tecnicoResponsable": "Carlos López",
  "tiempoEstimadoHoras": 3,
  "tipoMantenimiento": "Cambio de aceite"
}
```

La primera prueba consistió en la creación de un reporte de mantenimiento enviando una solicitud POST con todos los datos requeridos. En esta solicitud se incluyen los atributos obligatorios del reporte, como la placa del vehículo, el kilometraje, la fecha y el tipo de mantenimiento. Además, se pueden incluir atributos opcionales como observaciones, prioridad, técnico responsable, taller, costo estimado o nivel de desgaste. Una vez recibida la solicitud, el sistema utiliza el patrón Builder dentro del servicio ReporteMantenimientoService para construir el objeto ReporteMantenimiento paso a paso y finalmente generar la instancia completa mediante el método build().



```
POST http://localhost:8080/reportes/mantenimiento

{
  "placaVehiculo": "ABC123",
  "kilometraje": 120000,
  "fecha": "2026-03-15",
  "tipoMantenimiento": "Preventivo"
}
```

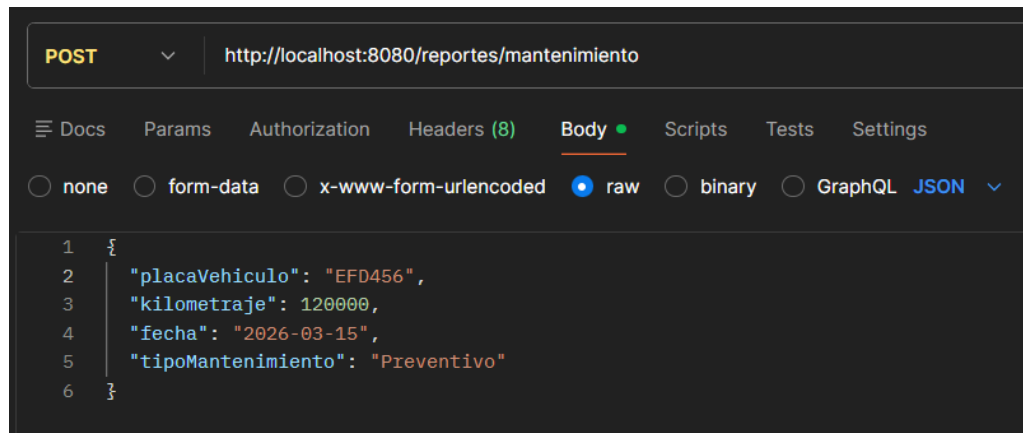
```
Body Cookies Headers (5) Test Results

{ } JSON Preview Visualize

1 {
2   "costoEstimado": null,
3   "fecha": "2026-03-15",
4   "kilometraje": 120000.0,
5   "nivelDesgaste": null,
6   "observaciones": null,
7   "placaVehiculo": "ABC123",
8   "prioridad": null,
9   "proximoMantenimientoKm": null,
10  "requiereRepuestos": null,
11  "taller": null,
12  "tecnicoResponsable": null,
13  "tiempoEstimadoHoras": null,
14  "tipoMantenimiento": "Preventivo"
15 }
```

Otra de las pruebas realizadas fue la creación de un reporte sin diligenciar algunos de los atributos opcionales. En este caso, el sistema permitió crear el reporte correctamente, lo cual demuestra el funcionamiento del patrón Builder, ya que permite construir objetos complejos

separando los atributos obligatorios de los opcionales, evitando la necesidad de utilizar constructores extensos con múltiples parámetros.



```
POST http://localhost:8080/reportes/mantenimiento

Docs Params Authorization Headers (8) Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 {
2   "placaVehiculo": "EFD456",
3   "kilometraje": 120000,
4   "fecha": "2026-03-15",
5   "tipoMantenimiento": "Preventivo"
6 }
```

```
java.lang.RuntimeException: El vehículo con placa EFD456 no existe
```

Adicionalmente, se realizó una prueba en la cual se intentó crear un reporte de mantenimiento utilizando una placa de vehículo que no se encuentra registrada en la base de datos. En este caso, el sistema realiza una validación previa dentro del servicio `ReporteMantenimientoService`, consultando el repositorio de vehículos. Si el vehículo no existe, se genera una excepción y el sistema no permite crear el reporte. Esta validación garantiza la integridad de la información, evitando que se registren mantenimientos asociados a vehículos inexistentes.

CONCLUSIONES

El desarrollo del Sistema de Gestión de Flotas permitió aplicar de manera práctica diversos conceptos fundamentales de la ingeniería de software orientada a objetos, demostrando cómo una adecuada organización arquitectónica puede contribuir a la construcción de sistemas más mantenibles, escalables y desacoplados. A través de la implementación de la arquitectura hexagonal, fue posible separar claramente la lógica del dominio de las dependencias tecnológicas externas, garantizando que las reglas del negocio permanezcan independientes de frameworks, bases de datos o interfaces externas.

La utilización de esta arquitectura permitió estructurar el sistema en diferentes capas bien definidas, como dominio, aplicación e infraestructura, facilitando la comprensión del flujo de funcionamiento del sistema y promoviendo el principio de inversión de dependencias. Gracias a este enfoque, el núcleo del sistema se mantiene protegido de cambios tecnológicos, permitiendo que la aplicación pueda evolucionar o adaptarse a nuevas tecnologías sin afectar la lógica principal del negocio.

Asimismo, la implementación de distintos patrones de diseño creacionales, como Singleton, Factory Method, Abstract Factory y Builder, permitió mejorar significativamente la organización del código y la gestión de la creación de objetos dentro del sistema. El patrón Singleton fue utilizado mediante el contenedor de Spring para garantizar una única instancia de los servicios y repositorios, optimizando el uso de recursos y asegurando una gestión centralizada de la lógica del sistema.

Por su parte, el patrón Factory Method permitió delegar la creación de diferentes tipos de vehículos a fábricas especializadas, evitando el uso de estructuras condicionales complejas dentro de los servicios y reduciendo el acoplamiento entre componentes. De manera complementaria, el patrón Abstract Factory permitió organizar la creación de familias de objetos relacionados, garantizando que los distintos tipos de vehículos puedan ser generados de forma flexible y extensible dentro del sistema.

Mientras que, el patrón Builder fue utilizado para la construcción de reportes de mantenimiento, permitiendo crear objetos complejos de manera progresiva y estructurada.

Este patrón facilitó la separación entre atributos obligatorios y opcionales, mejorando la legibilidad del código y evitando la necesidad de utilizar constructores extensos con múltiples parámetros.

Las pruebas realizadas mediante herramientas como Postman permitieron validar el correcto funcionamiento de los servicios implementados, confirmando que el sistema puede gestionar correctamente la creación, consulta, actualización y eliminación de vehículos, así como la generación de reportes de mantenimiento asociados a los mismos. Estas pruebas evidenciaron que la arquitectura y los patrones de diseño aplicados cumplen adecuadamente su propósito dentro del sistema.

En conclusión, el desarrollo de este proyecto permitió demostrar cómo la aplicación adecuada de principios de diseño, arquitecturas desacopladas y patrones de diseño contribuye a la construcción de sistemas de software más robustos, organizados y fáciles de mantener. Este tipo de enfoques resulta fundamental en el desarrollo de aplicaciones modernas, especialmente en entornos donde los sistemas deben evolucionar constantemente para adaptarse a nuevas necesidades tecnológicas y de negocio.

REFERENCIAS BIBLIOGRAFICAS

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
<https://www.oreilly.com/library/view/effective-java-3rd/9780134686097/>

Cockburn, A. (2005). Hexagonal architecture. Recuperado de
<https://alistair.cockburn.us/hexagonal-architecture>

Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley Professional. <https://domainlanguage.com/ddd/>

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
<https://martinfowler.com/books/ea.html>

Freeman, E., Robson, E., Sierra, K., & Bates, B. (2020). Head first design patterns (2nd ed.). O'Reilly Media. <https://www.oreilly.com/library/view/head-first-design/9781492077992/>

Gamma, E., Helm, R., Johnson, R., & Vlissides, R. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley. <https://www.oreilly.com/library/view/design-patterns-elements/0201633612/>

Gamma, E. (1995). Design patterns: Abstraction and reuse of object-oriented design. European Conference on Object-Oriented Programming.
<https://link.springer.com/chapter/10.1007/BFb0053381>

Oracle. (2023). The Java tutorials: Object-oriented programming concepts. Oracle Corporation.
<https://docs.oracle.com/javase/tutorial/java/concepts/>

Refactoring Guru. (2024). Design patterns catalog.
<https://refactoring.guru/design-patterns>

Richardson, C. (2018). Microservices patterns: With examples in Java. Manning Publications.
<https://microservices.io/book>

Spring. (2024). Spring Boot reference documentation.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (2024). Spring framework reference documentation.
<https://docs.spring.io/spring-framework/docs/current/reference/html/>